

AssistRx IDP: Project Summary and Key Requirements

Oct 5, 2026 · @Josh Smitherman

Summary

IDP (Intelligent Document Processing) is an AssistRx initiative to automate intake of faxed and scanned patient documents: classify every page, split packets by patient and document type, extract fields, and hand structured data to downstream systems without a person keying it in.

Product was running a buy-versus-build evaluation with Salesforce/MuleSoft IDP as the lead vendor. The Chief Product and Technology Officer (Lynn) leans build, so the AI team was asked to show an in-house solution. Josh rebuilt the earlier CMS-1500 form processor into a prototype that classifies pages and extracts enrollment-form fields with three LLMs running in parallel.

Where it stands across these meetings:

- Extraction and classification work on roughly 800 real PDFs from the Madrigal program. Reported field accuracy on enrollment forms ranges from 88% to 96% depending on the sample and ground truth used.
- Product's position is that extraction is table stakes. The open gap is orchestration: splitting, routing, human review, and integration with the submission API and case management.
- The requirements document is broad and partly open-ended. Cliff and Josh proposed phasing it; product agreed enrollment forms and copay claims come first.

Latest developments:

- **Bake-off.** The in-house prototype and Salesforce MuleSoft will each process a new set of documents, and accuracy will be compared. Both overall accuracy and accuracy on the fields the system is most sure about will count.
- **Build sizing.** Product mapped the target end-to-end process and is collecting build hours per step to set against vendor pricing. About 370 to 410 hours are estimated so far, with several steps still unsized.

- **Vendor shortlist.** Zach said Salesforce and one other vendor are the only ones being taken seriously alongside the in-house build, because both split and regroup packets by patient.

This doc is compiled from 22 Otter transcripts. Speaker labels were mostly anonymous, so attributions are inferred from context, and name spellings follow the transcripts and may be wrong. Two transcripts are mostly unrelated to IDP and are marked in the meeting index.

Key requirements

These are the requirements stated or clarified in the meetings. The source of record is product's business requirements document (owner: Zach), including Appendix A (enrollment form fields) and Appendix B (document types), which was discussed but not included in the transcripts.

Scope and phasing

- **Phase 1:** classify pages into as many document types as there are examples for, and do field-level extraction on patient enrollment forms only. Copay claims are the next extraction target.
- **Why these two:** product said enrollment forms and copay claims make up most fax volume and most manual data entry. Most other document types are only received and attached to a case, so classification matters more than extraction for them.
- **End state:** about a dozen standard document types (Appendix B), one central intake application for all inbound faxes, email attachments and file drops, usable by more than one product (CRM first; Scriptly, Momentum and next-gen CaseAssist were named).

Functional requirements

#	Requirement	Detail from the meetings
1	Handle scanned input	Assume every document is a scan or fax with no embedded form fields. Handwriting is expected to be the majority on enrollment forms.
2	Handle large files	Process 100+ page files without rejection or truncation. 20 to 25 pages is routine; 400-page faxes occur. Breaking point is untested.

#	Requirement	Detail from the meetings
3	Classify every page	Each page gets a known type, including "other" and "non-data page". No page is skipped.
4	Copay claim definition	A copayment claim is a CMS-1500 form (confirmed by Zach, who supplied a sample).
5	Split packets	Split a fax by patient and by document type, and keep the patient link through delivery. A 20-page fax with two patients, one with three document types, must come back as four documents grouped under two patients. Each page belongs to one patient, but continuation pages may not repeat the name, and a cover page may apply to several patients. Split documents get new DocMan IDs that trace back to the original.
6	Store document types separately	Needed for privacy and data sharing. A financial hardship form attached to an enrollment form recently bypassed privacy filters.
7	Extract enrollment form fields	Fields listed in Appendix A. Output is JSON. Product is to mark which fields are critical.
8	Detect signatures	Report present or absent for patient, provider and representative signatures.
9	Flag handwritten vs typed	Per field, so accuracy can be reported for each and compared with vendor claims.
10	Confidence score per field	Return and store the actual score, not only pass or fail against a threshold. Thresholds may differ by field.
11	99%+ accuracy above threshold	Clarified by Zach: not 99% on all fields. When a field passes the confidence threshold, it should be right 99%+ of the time and can skip human review.

#	Requirement	Detail from the meetings
12	Human review routing	Send only low-confidence, model-disagreement or failed-validation fields to a person. Expect everything to be reviewed at launch, then dialed back. Thresholds are configurable. Review also covers page grouping the system was unsure about, and lets the reviewer regroup the packet.
13	Missing information	Identify required fields that are blank (signature, consent, gender and similar) so a request can go back to the prescriber. Some vendors send that notification automatically; product wants it scoped but not in a day-one build.
14	Combine across pages	When the form is blank but an attached insurance card or loose page has the data, use it. Josh saw this in an estimated 15 to 20% of submissions.
15	Auditability	Trace every value to the page and document type it came from, especially when values are combined.
16	Process at intake	Run before a person opens the document. The old CMS-1500 flow ran only when a user clicked a button in Salesforce, and users found copy-paste faster.
17	Integrate downstream	Deliver to the submission API and case management, with routing to the right Salesforce org. Create a patient record or attach to an existing one, and assign the case owner a task saying a document arrived. Data enrichment from master data happens in the submission API: HCP today, site of care and patient later.
18	Match existing patients	Most forms carry no patient ID, so matching needs name, date of birth and similar fields, with MDM as a reference. Rules are needed for which fields may be overwritten on an existing case.

Target process and build estimates

Product's process map for the buy-versus-build analysis. Hours are remaining build effort as discussed; the session ended before every step was sized.

Step	What happens	Build estimate
1	A prescriber faxes a dedicated program fax number. The fax system sends images to the DocMan API, which stores them.	0 hours, exists today
2	An orchestration layer is alerted, retrieves the document with its ID and the org or workflow ID, and manages every hand-off below.	100 hours (Steve)
3	The extraction service classifies each page and extracts key fields.	120 hours remaining (Josh), up to 160 if patient grouping sits here
4	Pages are grouped by patient and document type. Agreed in the meeting that this comes after classification and extraction, not before.	Owner undecided
5	Missing key data is flagged, with an optional notification back to the prescriber.	Not sized, not day one
6	Fields and groupings below the confidence threshold go to a review platform, where a person corrects and resubmits.	150 hours (Steve), depends on the tool chosen
7	Orchestration requests new DocMan IDs for split documents and assembles the JSON with source documents.	Partly covered by step 2
8	The submission API enriches the data and routes it to the right case management system.	Not sized
9	Case management creates or updates records and assigns a task to the case owner.	Assumed to exist

Non-functional requirements

- **PHI handling:** only US data zone model deployments may process PHI. Global standard deployments are not allowed without an exemption. The Opus and Sonnet deployments used in the prototype were global standard and must be swapped.
- **Latency:** the requirements document has a processing-time requirement. The figure was not stated in the meetings.
- **Cost:** report cost and token use per page, by model, so the build can be compared with vendor per-page pricing.
- **Measurement:** keep a standing accuracy pipeline with human-verified ground truth, and track accuracy as models and prompts change.
- **Generic prompts:** one prompt set per document type, not a template per brand or manufacturer. Per-field instructions of two to three sentences, a grounding prompt for global rules, and long written criteria for classification.
- **Single output layer:** the extractor should write to one place that consuming systems read from, not integrate separately with each application.

Requested beyond the document

- Validation as a separate service after extraction, for example checking NPI and prescriber name against the CMS NPI registry.
- Model drift monitoring over time.
- Form-specific notes where a form is ambiguous, for example which of two refill numbers to take on the Rezdiffra form. Now handled by extracting bridge and non-bridge prescription fields separately.

Solution as built

The prototype is a single local endpoint that takes one PDF and returns classified pages and extracted fields as JSON. It started from the CMS-1500 form processor code; Josh estimated about half of that code remains.

1. **Intake.** One PDF per request. A batch script loops over files for testing. Source PDFs are roughly 800 Madrigal documents in blob storage.
2. **Page classification.** Each page is classified as patient enrollment form, insurance card, copayment claim, other, or non-data page. Supervised classifiers (logistic regression, random forest, gradient boosting, nearest neighbor) are trained on tokenized page text from manually labeled pages.

3. **Field extraction.** Each page is sent as an image to three vision LLMs in parallel: GPT, Claude Opus and Claude Sonnet. Azure Document Intelligence OCR was tested and removed because it performed far worse.
4. **Combine.** Results from the three models are merged, because each leaves different fields blank. Disagreements are kept and shown.
5. **Annotate.** Each field carries a confidence score per model, a handwritten flag, and the page it came from. Signature presence is reported. Confidence is now the model's actual per-field score rather than a fixed value. Bridge and non-bridge prescription fields are extracted separately, which stopped a blank section being filled with a default refill value.
6. **Ground-truth lookup (evaluation only).** The patient ARX ID is taken from the file name and used to query Snowflake for what staff entered. This checks accuracy; it does not feed extraction.
7. **Report.** A spreadsheet per run with one row per document and field, each model's value and confidence, the ground truth, and match or miss.

Design decisions made along the way:

- **Vision LLM over OCR.** Matches what Salesforce described, and tested better than Document Intelligence.
- **Generic prompts over per-form templates.** An early idea was a library of brand-specific forms with mapped field locations and a routing agent. This was dropped after the Salesforce demo showed generic per-field prompts working across forms.
- **Multi-model committee.** Costs more tokens. Kept because it improves completeness and gives a second signal for confidence. Salesforce called dual-model runs inefficient but reasonable for high-stakes fields.
- **Extraction and validation are separate services.** Extraction reports what is on the page. Lookups, reconciliation and review rules come after.

Not built yet: grouping pages by patient and document (confirmed as required in the process-mapping call), cross-page fusion, human review interface, any integration with the submission API or Salesforce, automatic triggering, cost and token reporting, and field extraction for anything other than enrollment forms. The page-count limit is untested.

Josh estimated two and a half to three weeks (120 hours) to extend classification, turn the prototype into a usable API and test it. The current output is one record per page with its type and any patient name found; returning results grouped by patient is new logic and was not in that figure.

Evaluation and accuracy

The reported numbers are not directly comparable: samples, models, prompts and ground truth changed between meetings. The transcripts are undated, so the order below is inferred.

Stage	Sample	Page classification	Field accuracy	Notes
First demo	7 synthetic PDFs with form fields	About 85% precision, recall, F1	Not measured	Logistic regression classifier. Cliff asked for scan-only inputs.
Scan-style synthetic	7 synthetic PDFs, no form fields	96% per page	76%	GPT-4 vision with basic prompts. Document Intelligence tested much worse.
First real-data runs	50 manually reviewed documents	76% overall, about 95% excluding copay	About 94% combined models	Copay pages misclassified with high confidence.
Product demo	50 audited documents	99% on enrollment forms	About 96%	Copay still weak. Insurance card lower, with only about 12 training examples against 75+ for enrollment.
Latest manual set	63 manually evaluated documents	Enrollment 84%, copay 96%, insurance card 79%	88.1% on enrollment forms	Older model set, not broken out by model. Josh's summary range: 85 to 95% classification, 88 to 90%+ field.

A 30-document run scored against Snowflake values, broken out by model, was in progress at the last meeting.

Salesforce benchmark

- About 96% of fields correct, 97.1% on handwritten and 100% on typed.
- Tested on created forms, not real patient documents. Each document was run 10 to 20 times and results averaged.
- Models: Gemini Flash 3.5 preferred, Claude Sonnet 4.6 second. Gemini is not available on Azure.
- No classification accuracy data and no per-field accuracy data were available.
- Confidence comes from the LLM's own score, with 80% as the default review threshold. Their claim: every wrong field also had low confidence.

Ground truth

- **Manual review** is the trusted source: LLM output on one screen, document on the other, field by field.
- **Snowflake** values are used at scale, keyed on patient ARX ID. A patient can have several records, so a field counts as a match if it equals any stored value. Not every field or document has data, and some stored values are cleaned up versions of what was written (payer names, for example), so a correct read of the form can score as a miss.
- Cliff's concerns: labels generated by an LLM should not be used to grade an LLM, and accuracy against manual entry should stay available alongside the Snowflake comparison.
- Jason's advice from the CMS-1500 work: define ground truth as what is written on the form (for example the word "same"), keep reconciliation as a later step, and run a standard accuracy report on every commit.

Known failure patterns

- Handwritten identifiers such as NPI, policy number and Rx group number are misread, sometimes with high confidence. Josh reported all three models making the same NPI error; Cliff asked him to re-check that.
- GPT returns the most blanks. The two Claude models usually agree with each other.
- Non-enrollment pages that mention enrollment (for example a benefit investigation summary) get classified as enrollment forms.
- Data needed for the form sits on a page classified as something else, which lowers measured enrollment-form accuracy.

Bake-off against Salesforce

New documents will be run through both the in-house prototype and MuleSoft IDP. Cliff's instruction: focus on accuracy and on knowing when the system is accurate, and set aside user-facing prompt tooling for now.

Evaluation set agreed:

- 50 documents: 25 handwritten and 25 typed, each group randomly selected. A plain random 50 would not work, because the first manual set of 50 had only about two handwritten documents.
- Each document is run through the pipeline, Snowflake values are appended, then Josh verifies every field by hand.
- Two accuracy measures are kept side by side: against Snowflake, and against the hand-verified values.
- The share of documents that are handwritten is reported as well.

Metrics to report:

- Overall field accuracy, per model and combined.
- Accuracy within confidence bands, for example fields at 90% confidence and above, then 80% and above, with the count of fields in each band.
- Accuracy when all models agree, when they disagree, and when some return blank.
- Fields with the highest model disagreement, such as NPI, to target prompt fixes or flag unreadable inputs.

Model selection:

- Only models deployed in US East or US East 2 with data zone availability can be used, which rules out most public OCR leaderboard entries.
- Plan: export the list of models available in Azure Foundry and shortlist from it for image-to-field extraction. Mistral Document AI is the first candidate to add.
- Josh's impression is that Claude Sonnet returns the most complete results; this is not yet backed by numbers.
- Unresolved: whether reasoning effort can be set through the API for these models.

Earlier analysis requests

- A calibration table: confidence bucket, count of fields, accuracy in that bucket.
- Accuracy when all models agree versus when they disagree or some return blank.

- Share of fields that can skip human review at a given threshold, and the accuracy of that group.
- Cost per page by model, to set against accuracy.
- Accuracy by page count, and per-field accuracy.

Data, access and environment

Access was the main source of delay early on. By the later meetings Josh had the resource group, the PDFs, and newer models.

Area	What the meetings established
Development environment	An Azure VM reached through Zscaler with an admin domain account. The password rotates and is retrieved from Secret Server with authenticator sign-in. The old repo is the AI form processor for CMS-1500.
Azure subscription	Reuse the subscription and resource group already vetted for PHI from the CMS-1500 work. Setting up a new one was described as slow.
Source documents	Madrigal documents are exported once to the program's outgoing blob container, under a documents folder. Access needs its own ticket.
Finding a document	A Snowflake query on the Salesforce document files table returns document type (one value is patient enrollment form) and a file name that matches the blob name. It joins to the patient table.
Ground-truth data	Read access to the Snowflake presentation layer including PHI, approved by Michelle Mullins. Some PHI is masked. A service account is needed once comparisons run as a job.
Models	Started on the legacy GPT-4 deployment. Later GPT-5, Claude Opus 5 and Claude Sonnet 5. Gemini is not on Azure. US-zone deployments Jason pointed to include Sonnet 5 US, Opus 4.8 US and GPT 5.2 US. Mistral Document AI was flagged as worth testing.

Area	What the meetings established
Model deployment	Josh cannot deploy models himself. Matt Weber does it. Moving a deployment from global standard to data zone standard means deleting and recreating it; the API key stays the same and the endpoint changes.
Usage and cost data	Token and cost reporting needs billing permissions Josh lacks, and guardrails on token use need an extra Azure service. Jason warned this is a bigger lift than it sounds.
Source control	A new Azure repo for IDP was requested by ticket and assigned to Matt. IDP should not share the CMS-1500 repo.
Program list	A program inventory sheet lists drugs and programs. Filter to live programs where CaseAssist is marked or hub is internal to find forms AssistRx actually processes.

Lessons on getting access:

- Josh's subscription access was held by compliance until overdue KnowBe4 training was complete. Nobody had told him.
- Tickets stall unless followed up by phone. Alex helped escalate; Kareem (DevOps) and Becky Nichols (compliance) cleared the block.
- Jason warned that connecting a deployed app to blob storage and Snowflake took longer than expected on the CMS-1500 project because of policy and ownership boundaries.

People and roles

Roles are as described in the meetings. Spellings come from automated transcription.

Person	Role in this project
Josh	Builds the in-house IDP prototype. Staffed by Cliff to document processing initiatives.

Person	Role in this project
Cliff	Chief AI Officer. Sets direction, reviews progress, represents the build option.
Lynn (also transcribed Lin)	Chief Product and Technology Officer. Leans build and makes the final buy-versus-build call.
Zach Baker	Senior product manager and owner of the IDP requirements document.
Jeremy	Product, works with Zach on IDP and the vendor evaluation.
Jason	Built the original CMS-1500 form processor. Part-time; advises on models, PHI rules and lessons learned.
Steve (Stephen)	Handled the Salesforce side of the CMS-1500 processor and built the submission API with his team. Estimates the orchestration and review layers for the build.
John Mills	Data architect. Knows where enrollment data sits in Snowflake. A John Hawkins is also on the IDP team; the transcripts mix the two.
Matt Weber	DevOps. Deploys models and is creating the repo.
Alex	Helps push access tickets through.
Kareem, Becky Nichols	DevOps engineer and compliance manager who cleared the access block.
Michelle Mullins	Approves PHI access in Snowflake and pointed the team to the people who know the data.
Jordan Clark, Rory	System admin team for Snowflake access and service accounts.
Shravia, Chakshua, Harsh	Lead data architects named as alternates to John.
Alan	Product. Cliff asked that he be copied on next steps.
Tim, Kat	Lead the copay claims work with the vendor Infinity.

Person	Role in this project
Neville, Nikki	Proposed to help locate historical claim data and documents.
Chase Langdon, Ethan Choi, Matt, Janet	Salesforce and MuleSoft: account director, automation sales, solution engineer, lead automation architect.

Open questions and action items

These were open at the end of the transcripts. Some may have been closed since.

Questions for product and architecture

- Which Appendix A fields are critical, and what confidence threshold applies to each?
- What is the latency target, and is it per page or per document?
- How is processing triggered: file drop with a listener, the document management API, or a direct API call?
- Who consumes the JSON output, and in what payload shape? Is the submission API the single entry point?
- Does patient grouping live in the extraction service or the orchestration layer? The process-mapping call ended before this was settled.
- What does human review look like in Salesforce, and can it show only the flagged fields?
- What are the rules for combining data across pages and for overwriting fields on an existing patient?
- Where are examples of the other Appendix B document types (clinical notes, labs, prior authorization, EOB)? None have turned up in the 800-document sample.
- What share of enrollment forms is handwritten? Product believes most, with no data.
- What is the full list of systems that need IDP output? Josh proposed mapping the end-to-end architecture to force these decisions.
- How should pages with no patient name be grouped: page order, page numbers, or other context? And what happens to a cover page shared by several patients?
- Where is the extraction JSON stored and how is it linked to the original and split DocMan IDs? DocMan and Snowflake were both raised.

- Which steps of the target process still need an estimate, and who owns each?

Actions

- ☐ Move Claude and GPT deployments to US data zone endpoints and update the prototype (Josh, Matt Weber)
- ☐ Track model agreement as a column alongside the per-field confidence scores now stored (Josh)
- ☐ Produce the confidence calibration table and the human-review split (Josh)
- ☐ Re-check whether all three models really return the same wrong NPI (Josh)
- ☐ Grow the manually verified set and balance it across document types (Josh)
- ☐ Add CMS-1500 specifics to the copay classification criteria, and lengthen classification instructions generally (Josh)
- ☐ Test very large files and report accuracy by page count (Josh)
- ☐ Get cost and token usage per page by model (Josh, needs billing access)
- ☐ Finish repo setup and push the code (Josh, Matt Weber)
- ☐ Turn the requirements document into phases, and mark critical fields (Zach, with Cliff and Josh pushing)
- ☐ Draft the end-to-end architecture and a list of answered and unanswered requirements (Josh)
- ☐ Keep a running list of features built and not built (Josh, requested by Cliff)
- ☐ Email next steps to Cliff and copy Alan (Josh)
- ☐ Get Salesforce's added test forms, model outputs, ground truth and reasoning-level settings (Cliff, from Salesforce)
- ☐ Evaluate Mistral Document AI and current Azure vision OCR options (Josh, Jason)
- ☐ Set up drift monitoring and a per-commit accuracy report (Josh)
- ☐ Build the 25 handwritten plus 25 typed evaluation set and hand-verify every field (Josh)
- ☐ Write up the bake-off metrics plan and send it to Cliff, then send daily performance updates (Josh)
- ☐ Export the available Azure model list and shortlist models for image-to-field extraction (Josh)
- ☐ Build patient and document grouping logic, once its home is decided (Josh or orchestration owner)
- ☐ Reorder the process map so classification and extraction come before splitting (Zach)

- ☐ Reconvene to finish the build estimates (Zach, Steve, Josh, Cliff)

Related workstreams

- **Buy versus build.** Salesforce prices per page processed, with classification and extraction counted as two runs, plus integration flows. Vendors quoted six to eight weeks. Zach estimated about six months to build the full pipeline in-house and expected a start in late Q4 or early Q1. Build cost is now being sized step by step; see Target process and build estimates.
- **Copay claims and Infinity.** AssistRx is going live with Infinity for copay claim processing in mid-November at \$12 per transaction, with 50,000 to 60,000 claims a year expected for one client. An in-house backup was scoped as three projects: CMS-1500 extraction, EOB extraction, and adjudication logic. CMS-1500 fits the IDP pipeline; EOB is harder because layouts and column terms vary by payer.

Meeting index

Grouped by phase of the project. The order within and across groups is inferred from content.

Transcript	Who	What it covered
Kickoff and access		
Document Processing Setup Meeting	Josh, Cliff	Confirmed the CMS-1500 processor still runs. VM and Secret Server access. Could not find the blob storage. Goal set: a library of enrollment forms and extraction.
Pharmaceutical Form Processing Meeting	Josh, Cliff	Program inventory sheet and how to filter it. Early idea of form-specific agents. Tickets for subscription and Snowflake access.
Snowflake API Setup Meeting	Josh, a data platform lead	Who to ask about Snowflake data and access. Service account needed for comparisons.

Transcript	Who	What it covered
Enrollment Data Processing Meeting	Josh, a data engineer	Where Madrigal documents sit in blob storage and the query to match them to Snowflake records.
Synthetic-data prototype		
Form Processing Strategy Meeting	Josh, Cliff	Few-shot prompts, signature detection, validation as a separate service, local development. Second half is about a job candidate.
Document Classification Demo	Josh, Cliff	First demo on 7 synthetic documents. Assume scans. Requirements document shared; vendor meetings to join.
Salesforce Document Automation Meeting	Salesforce team, Cliff, product	Salesforce IDP demo: four-step flow, generic prompts, confidence scores, POC results, pricing basis.
Document Intelligence Evaluation Meeting	Josh, Cliff, Alex	Vision LLM beats Document Intelligence. Salesforce's models and prompts. Access blocked by overdue compliance training.
AI Model Accuracy Analysis	Josh, Cliff	Cost per page by model, LLM confidence scores, two-model comparison as a review signal.
Real data		
Document Classification Process Review	Josh, Cliff	First run on real PDFs with GPT-5, Opus 5, Sonnet 5. Ground-truth plan from Snowflake. Triggering options. Next-step list.
Patient Support Form Review	Josh, Cliff	Per-field and form-specific prompt notes. Requirements gap review. Human review thresholds. Drift monitoring.

Transcript	Who	What it covered
Model Optimization Strategy Meeting	Cliff, data science team, Josh	Mostly a different project (uplift models for call timing). Josh gives a short IDP update: 70 documents labeled, pre-meeting with Zach.
AI Process Evaluation Meeting	Josh, Zach, Jeremy	Phase 1 categories confirmed. Copay claim equals CMS-1500. Product explains that orchestration, not extraction, decides buy versus build.
Document Classification Review	Josh, Cliff	Copay misclassification. Calibration table requested. Plan to map the architecture to pin down requirements.
AI Document Processor Demo	Josh, Cliff, Zach, Jeremy, others	Demo to product. Splitting, signatures, the 99% clarification, critical fields, cross-page data, auditability.
Intelligent Document Processing Review	Cliff, Tim, Kat, Zach, Josh	Copay claims and EOB with Infinity. In-house backup scoped as three projects. Buy-versus-build timing.
Review with Jason		
IDP Project Review and Requirements	Josh, Jason	Project walkthrough. Phase 1 proposal. Jason flags that global-standard deployments cannot be used for PHI.
Model Deployment and Classification Review	Josh, Jason	US-zone models, classifier features, ground-truth philosophy, per-commit metrics, deployment lessons from CMS-1500.

Transcript	Who	What it covered
Model Accuracy and Performance Review	Josh, Cliff	Model agreement patterns. Latest accuracy numbers from the 63-document manual set. Feature list requested.
Bake-off and build sizing		
LLM Accuracy and Bake-off Planning	Josh, Cliff	Bake-off against Salesforce announced. Stratified 50-document evaluation set, two ground truths, metrics by confidence band and model agreement. Bridge and non-bridge prompt fix. Model shortlist approach.
Process Mapping for Buy vs Build Analysis	Zach, Steve, Cliff, Josh, Jeremy	Target end-to-end process walked through and sized in hours. Patient and document grouping confirmed as required and not yet built. Ended before all steps were estimated.
Not about IDP		
AI Model Evaluation Meeting	Josh, a colleague, a job candidate	Candidate interview about LLM evaluation tooling. Josh describes the document work in passing.